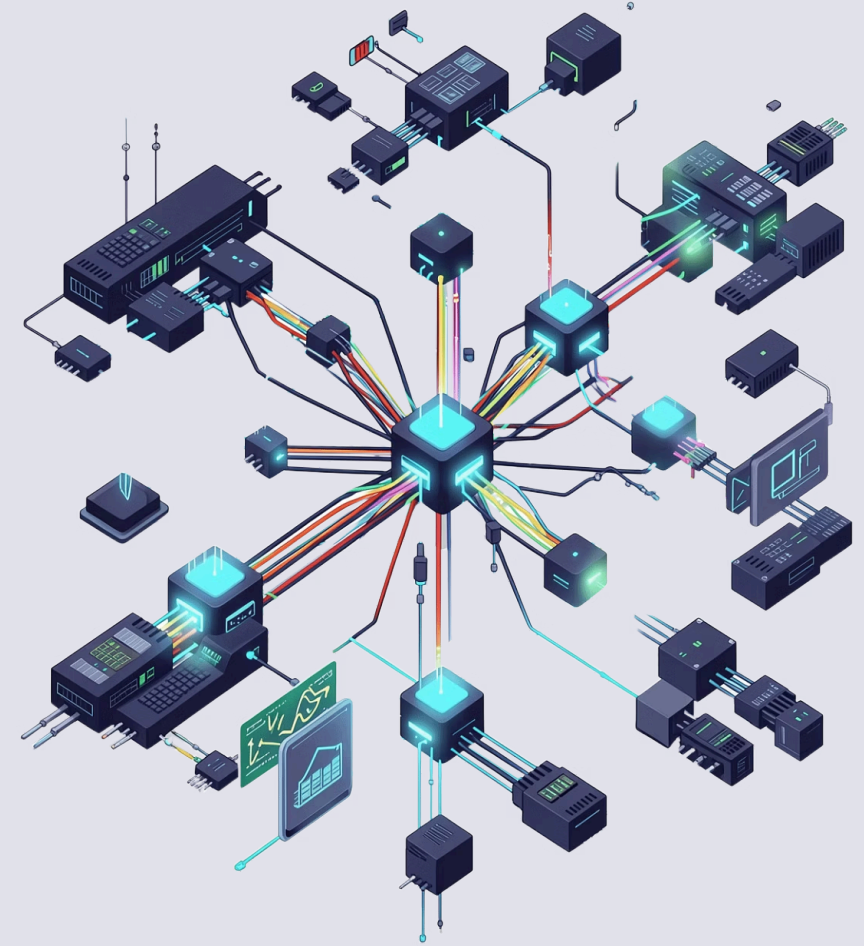


Multimodal Prompt Engineering & LLM Systems

Agentic AI & LLM Systems | B.Tech Computer Engineering

Dr. Anilkumar V. Brahmane | Department of Computer Engineering



What is Prompt Evaluation? Why Does It Matter?

What is Prompt Evaluation?

Prompt evaluation = **measuring how good your prompt is** at getting the right answer from an LLM.

A **bad prompt** → wrong, vague, or hallucinated output

A **good prompt** → accurate, useful response

Key Metrics

✓ Accuracy

Is the answer factually correct?

✓ Relevance

Does it answer what was asked?

✓ Consistency

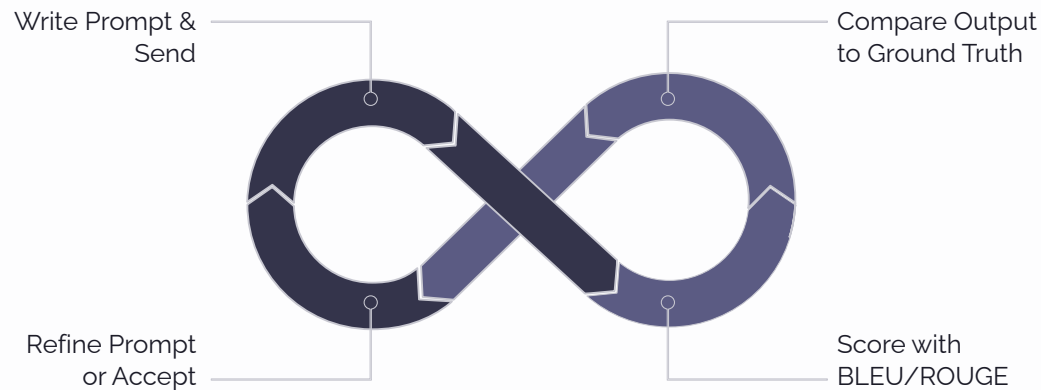
Does it give the same answer every time?

✓ Fluency

Is the language clear and natural?



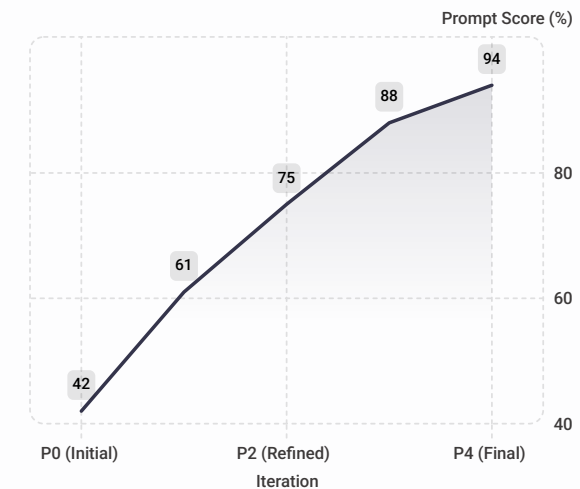
Prompt Evaluation: Flowchart & Algorithm



Each refinement cycle improves prompt quality until the score meets the acceptance threshold.

Algorithm: Iterative Prompt Refinement

1. Define task + expected output
2. Write initial prompt P_0
3. Run $LLM(P_0) \rightarrow \text{Output } O_0$
4. Evaluate: $\text{Score} = \text{Eval}(O_0, \text{Ground Truth})$
5. If $\text{Score} < \text{threshold} \rightarrow \text{Modify prompt} \rightarrow \text{Go to Step 3}$
6. Else $\rightarrow \text{Accept prompt as final}$



Prompt Evaluation: Python Implementation

Code: ROUGE Score Evaluation

```
from rouge_score import rouge_scorer
import openai

def evaluate_prompt(prompt, expected):
    # Get LLM response
    response = openai.chat.completions.create(
        model="gpt-4",
        messages=[{"role": "user", "content": prompt}]
    )
    output = response.choices[0].message.content

    # Score using ROUGE-L
    scorer = rouge_scorer.RougeScorer(['rougeL'])
    score = scorer.score(expected, output)
    print(f"ROUGE-L Score: {score['rougeL'].fmeasure:.2f}")
    return score

evaluate_prompt(
    "Explain photosynthesis in 2 sentences.",
    "Plants use sunlight to convert CO2 and water into glucose."
)
```

What This Code Does

- 1 Call LLM
Send the prompt to GPT-4 via OpenAI API and capture the output
- 2 Score Output
Use ROUGE-L to compare LLM output against the expected (ground truth) answer
- 3 Iterate
If score is low → refine your prompt and repeat the cycle

What is Hallucination in LLMs?

Hallucination: When an LLM generates text that sounds confident but is factually **WRONG** or completely made up.

Example: Ask GPT "Who won the 2030 FIFA World Cup?" → It may confidently give a fake answer.

Factual

Wrong facts — incorrect dates, names, or numbers

Contextual

Contradicts the given document or context

Logical

Reasoning errors in multi-step problems



Root Cause: LLMs predict the next likely word — they don't "know" facts. They pattern-match from training data.

Hallucination Mitigation Strategies



29/6

1

RAG

Retrieval Augmented Generation – fetch real documents and give to LLM as context. Grounds answers in facts, not memory.

2

Self-Evaluation

Ask the LLM to check its own answer – "Is this factually correct?" – using its internal knowledge.

3

Chain-of-Thought

Force step-by-step reasoning → reduces logical errors in complex tasks.

4

Temperature = 0

Set randomness to zero for factual tasks → more deterministic, consistent output.

5

Cite Sources

Instruct the model to cite sources for every claim it makes.

Hallucination Mitigation: Python Example (RAG)

Code: Simple RAG — Grounded Prompt

```
import openai

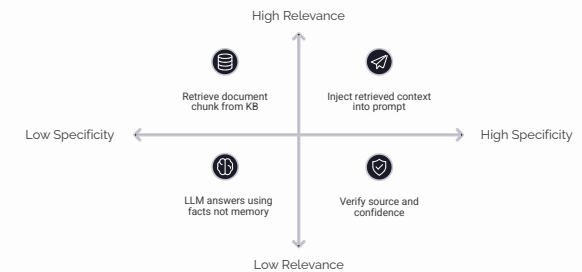
# Step 1: Your knowledge base (retrieved document chunk)
context = """
The Eiffel Tower is located in Paris, France.
It was built in 1889 and is 330 meters tall.
"""

# Step 2: User question
question = "How tall is the Eiffel Tower and where is it?"

# Step 3: Grounded prompt (RAG-style)
prompt = f"""Use ONLY the context below to answer.
Context: {context}
Question: {question}
Answer: """

response = openai.chat.completions.create(
    model="gpt-4",
    messages=[{"role": "user", "content": prompt}],
    temperature=0 # deterministic
)
print(response.choices[0].message.content)
```

How RAG Prevents Hallucination



✔ Setting `temperature=0` makes the output deterministic — ideal for factual Q&A tasks.

Multimodal Prompt Engineering

Multimodal: Working with multiple types of data — text, images, audio, video — all in one prompt.

Traditional vs Multimodal Prompt

Type	Input	Example
Traditional	Text only	"Describe a cat"
Multimodal	Image + Text	 + "What breed is this cat?"

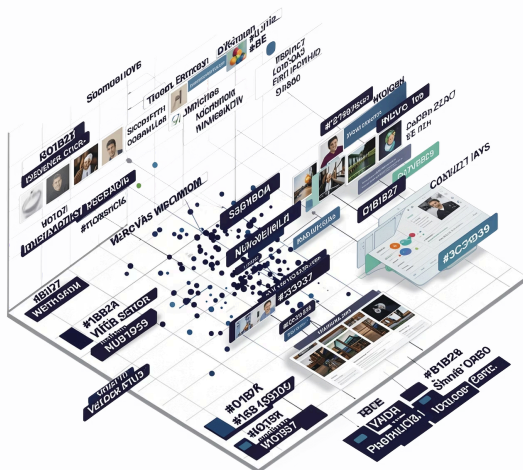
Chain-of-Modality Technique

Process one input first → use result to reason over the next input. This **cuts hallucination** versus dumping everything into one shot.

Models that Support This

- **GPT-4o** — OpenAI
- **Gemini 1.5 Pro** — Google
- **Claude 3** — Anthropic

Image & Text Embeddings — The Bridge Between Modalities



What is an Embedding?

Converting any data (text, image, audio) into a **list of numbers (a vector)** that captures its meaning.

Text Embedding

"King" $\rightarrow [0.2, 0.8, 0.1, \dots]$ "Queen"
 $\rightarrow [0.2, 0.7, 0.2, \dots]$

Similar vectors = similar meaning

Image Embedding

A photo of a dog $\rightarrow [0.5, 0.3, 0.9, \dots]$

Same vector space as text!

Key Idea

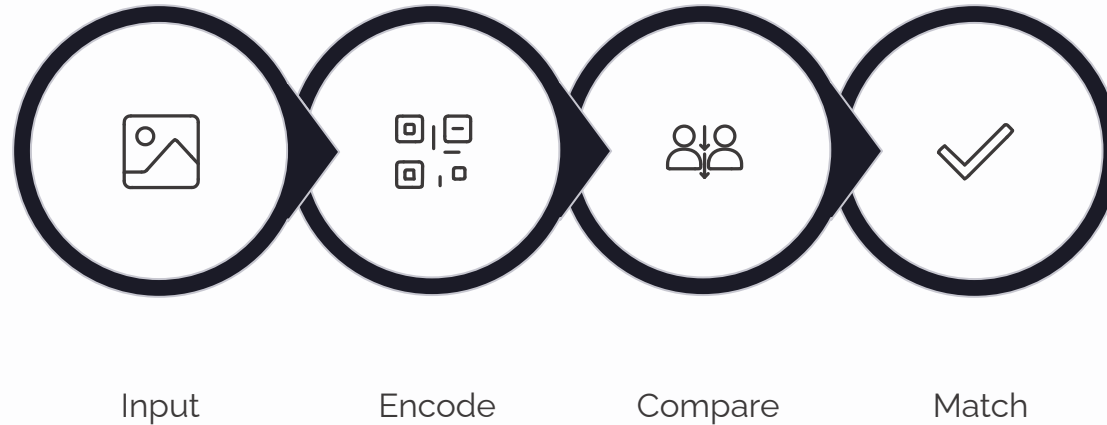
If text and image embeddings live in the **SAME vector space** → we can search across both modalities at once.

Used In

Semantic search, RAG, image
retrieval, recommendation systems

CLIP — Connecting Images and Text

How CLIP Works



CLIP = Contrastive Language–Image Pre-training

Developed by **OpenAI**. Learns to match images with their text descriptions by training two encoders simultaneously using contrastive loss.

Use Cases

- Zero-shot image classification
- Image search by text query
- Multimodal RAG retrieval

Trained on **400 million image-caption pairs** — text and image encoders output vectors in the **same shared space**.



Image Encoder

Converts image to vector



Text Encoder

Converts text to vector

CLIP: Python Implementation

Code: Zero-Shot Image Classification with CLIP

```
from PIL import Image
import requests
from transformers import CLIPProcessor, CLIPModel

# Load CLIP model
model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")

# Load an image
url = "http://images.cocodataset.org/val2017/0000000039769.jpg"
image = Image.open(requests.get(url, stream=True).raw)

# Text labels to compare
texts = ["a photo of a cat",
         "a photo of a dog",
         "a photo of a car"]

# Process inputs
inputs = processor(text=texts, images=image,
                  return_tensors="pt", padding=True)
outputs = model(**inputs)

# Get similarity scores
probs = outputs.logits_per_image.softmax(dim=1)
for label, prob in zip(texts, probs[0]):
    print(f"{label}: {prob.item():.2%}")
# Output: "a photo of a cat: 94.5%"
```

Step-by-Step Breakdown

01

Load Model

Load CLIP from HuggingFace — both processor and model weights

02

Load Image

Fetch any image from a URL or local file using PIL

03

Define Labels

Write text labels you want to match against the image

04

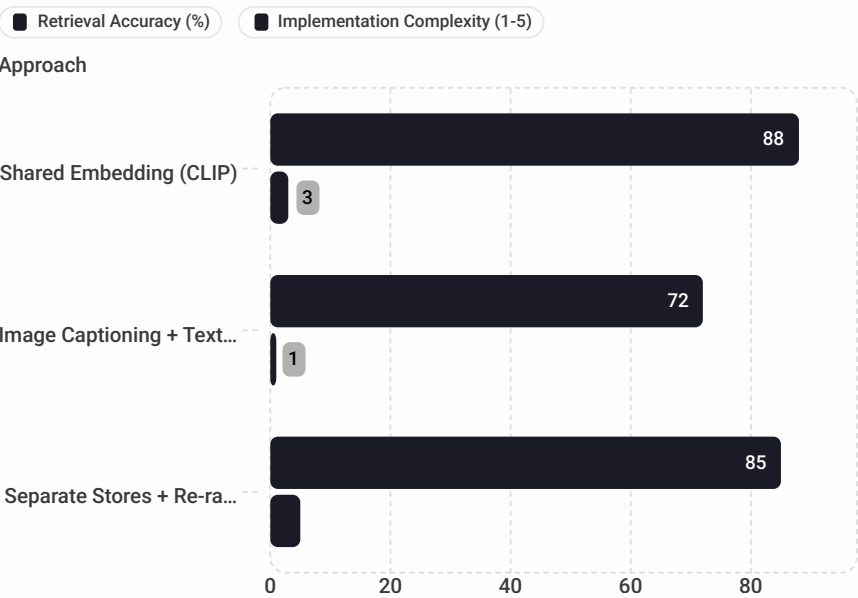
Compute Similarity

Softmax over logits gives probability score per label

✔ No training needed — CLIP does **zero-shot** classification out of the box!

Multimodal RAG — Searching Text AND Images Together

Three Approaches to Multimodal RAG



Approach 1 gives best retrieval accuracy but higher cost. Approach 2 is the simplest to implement for beginners.

Dr. Anilkumar V. Brahmane | Computer Engineering

Traditional vs Multimodal RAG

Feature	Traditional RAG	Multimodal RAG
Input	Text only	Text + Images + Audio
Retrieval	Text chunks	All modalities
LLM	Text LLM	Multimodal LLM

● Approach 1

Embed all modalities in same vector space (CLIP-style)

● Approach 2

Convert images → captions (text) → use standard text RAG

● Approach 3

Separate stores per modality + multimodal re-ranker

Processing Audio with Whisper



What is Whisper?

OpenAI's **open-source speech recognition model** — converts audio → text. Supports **99 languages**, transcription, translation, and timestamps.

In Agentic AI:

Audio Input → **Whisper** → Text → **LLM** → Response

Use cases: voice assistants, meeting summarizers, lecture transcription

Python Example: Transcribe + Summarize

```
import whisper

# Load Whisper model (tiny/base/small/medium/large)
model = whisper.load_model("base")

# Transcribe audio file
result = model.transcribe("lecture_audio.mp3")
print(result["text"])
# Output: "Today we will learn about neural networks..."

# Use transcription as LLM prompt
import openai
summary = openai.chat.completions.create(
    model="gpt-4",
    messages=[{"role": "user",
                "content": f"Summarize: {result['text']}"}]
)
print(summary.choices[0].message.content)
```

Gemini & GPT Multimodal APIs

Gemini Python Example

```
import google.generativeai as genai
from PIL import Image

genai.configure(api_key="YOUR_API_KEY")
model = genai.GenerativeModel("gemini-1.5-pro")

img = Image.open("diagram.png")
response = model.generate_content(
    ["Explain this diagram in simple terms.", img]
)
print(response.text)
```

📄 Gemini 1.5 Pro supports a **1 million token context window** – you can send entire books + images together!

Model Comparison

Feature	GPT-4o	Gemini 1.5 Pro
Text	✓	✓
Images	✓	✓
Audio	✓	✓
Video	✗	✓
PDF	✗	✓

What You Can Do

- 📷 Send an image → ask questions about it
- 🎵 Send audio → get transcription + analysis
- 🎬 Send a video → get scene-by-scene summary
- 📄 Send a PDF + image → cross-reference and answer

Putting It All Together — The Multimodal Agentic AI Pipeline

Full pipeline: User input (text / image / audio / video) → Embedding → Retrieval (RAG) → Multimodal LLM → Response



 CLIP

Image-text embedding & retrieval



 Whisper

Audio → text transcription



 Gemini / GPT-4o

Multimodal reasoning & generation



 RAG

Grounding answers in real documents



 Prompt Evaluation

Ensuring output quality



 **Next Steps:** Build your own multimodal RAG system using CLIP + Gemini API + a vector database (ChromaDB / FAISS)

Sources & References

Multimodal RAG with Gemini

Build a Q&A App with Multi-Modal RAG using Gemini Pro – Google Codelabs

Hallucination Mitigation

Self-Alignment for Factuality: Mitigating Hallucinations in LLMs via Self-Evaluation – ACL 2024

Multimodal Prompting Guide

How to Combine Image, Text, and Audio in One Prompt: A Multimodal Workflow (2026) – SurePrompts

Multimodal RAG in Production

Multimodal RAG in Production: When You Need to Search Images, Audio, and Text Together

Multimodal RAG Explained

Multimodal RAG – Intuitively and Exhaustively Explained – IAEE Substack